

AI-Driven Real-Time Interview Simulation and Recruitment System

Clear Project Reading Report for Review and Interview Explanation

Project Area	Details
Application Type	React + Vite frontend with Node.js + Express backend and MongoDB database
Core Purpose	To simulate interviews, evaluate answers, support coding practice, enable peer/recruiter interviews, and guide learning through AI coach and roadmaps
Major Browser Capabilities	Camera, microphone, face recognition, speech recognition, speech synthesis, WebRTC
Major Backend Capabilities	Authentication, OTP email verification, AI question/evaluation APIs, reports, roadmap generation, code endpoints, peer and recruiter room APIs

1. Problem Statement

Students and job seekers often prepare technical content but do not receive realistic practice in the same environment as an actual online interview. Traditional preparation is fragmented: one tool is used for video calls, another for coding practice, another for notes, and another for feedback. This makes it difficult to measure performance in a complete way.

Recruiters also face difficulty when they need to evaluate many candidates consistently. Manual interviews require time, subjective scoring, repeated scheduling, and separate tools for coding, resume viewing, video interaction, and feedback. Candidates may know the answer but still fail because they cannot communicate clearly, maintain confidence, or solve problems under time pressure.

The problem solved by this project is the absence of a unified platform that can combine AI mock interviews, voice-based answers, face monitoring, coding practice, peer mock interviews, recruiter interviews, career roadmaps, and final reports in one place.

2. Why This Project Was Taken

This project was selected because interview preparation is a practical and high-impact problem for students. Many students know programming concepts but are not comfortable explaining them in a live interview. A system that allows repeated mock interviews, speech practice, coding questions, and result analysis can improve readiness before real placement drives.

The project also demonstrates multiple important full-stack engineering concepts: authentication, secure backend APIs, database models, AI service integration, browser camera/microphone handling, speech recognition, face detection, real-time rooms, code editor integration, and deployment. Therefore, it is suitable for final review because it shows both practical usefulness and technical depth.

3. Proposed Solution

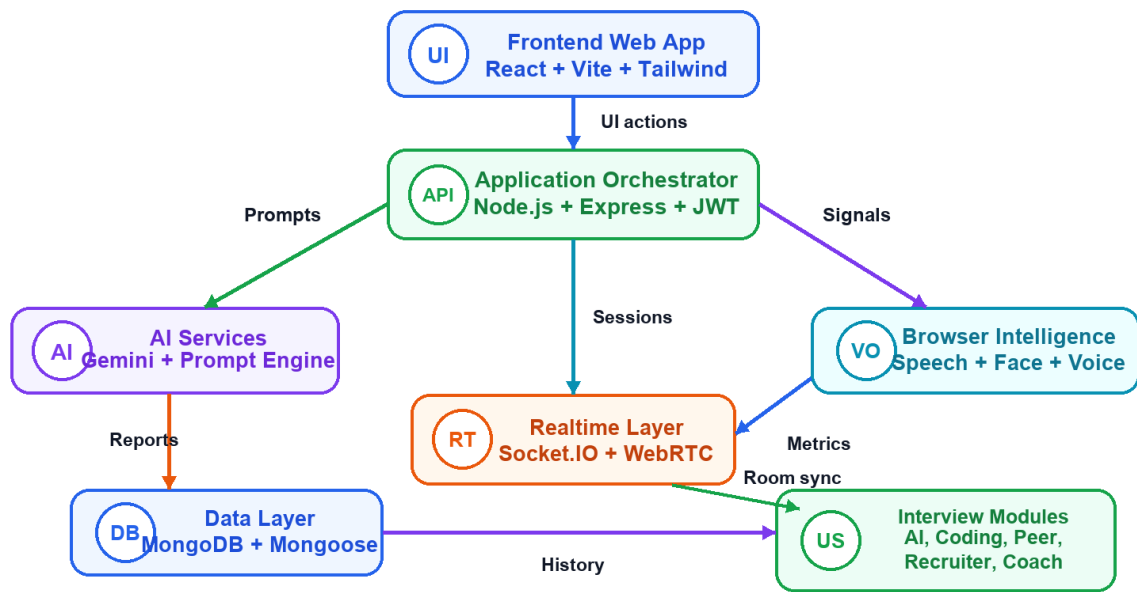
The proposed solution is a browser-based AI interview simulation and recruitment platform. A user can register, verify email, login, configure an interview, allow camera and microphone access, answer AI-generated questions, receive feedback, practice coding, participate in competitive coding battles, request peer mock interviews, join recruiter interview rooms, and use AI-based career guidance.

The system is not only an AI chatbot. It is a complete platform with multiple modules connected through a common dashboard, protected routes, backend APIs, MongoDB collections, and real-time features. AI is used where it adds value, while the backend still controls authentication, validation, storage, and workflow.

4. Overall Architecture

The project follows a layered full-stack architecture. The frontend handles the user interface, camera/microphone interaction, speech recognition, face detection status, and page routing. The backend handles REST APIs, authentication, AI calls, database writes, and server-side workflow. MongoDB stores persistent data. Socket.IO and WebRTC support real-time rooms for battle, peer, and recruiter features.

Architecture Diagram



Architecture diagram showing frontend, backend, AI services, browser intelligence, data layer, real-time layer, and interview modules.

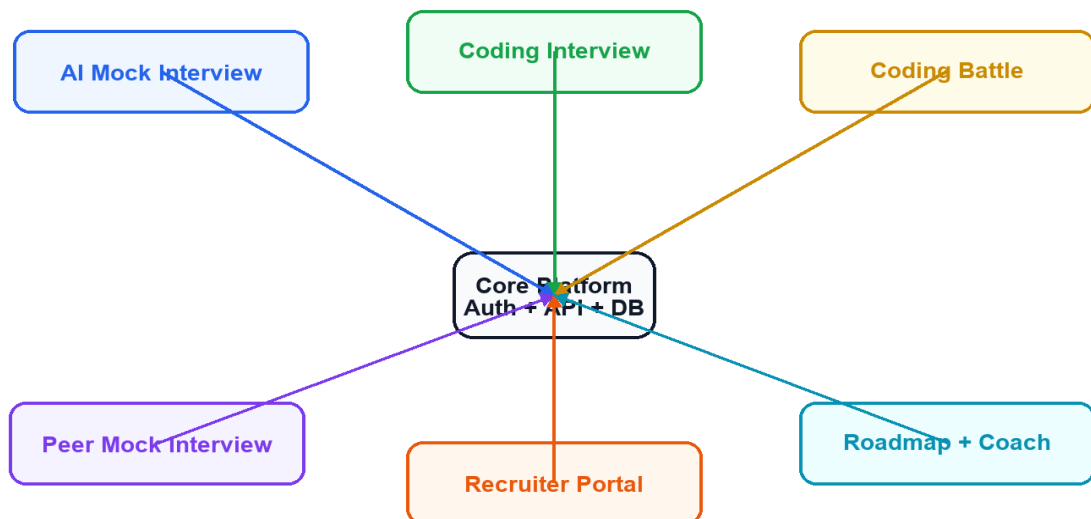
Layer	Technology	Responsibility
Frontend	React, Vite, Tailwind CSS, React Router	Pages, dashboard, interview UI, forms, code editor, reports
Backend API	Node.js, Express.js	Routes, controllers, authentication, validation, AI orchestration
Database	MongoDB, Mongoose	Users, interviews, resumes, roadmaps, chats, battles, peer/recruiter sessions
AI Layer	Gemini API / prompt services	Question generation, answer evaluation, AI coach, roadmap generation
Browser Intelligence	face-api.js, Web Speech API, Speech Synthesis	Face detection, voice transcription, interviewer speech
Realtime Layer	Socket.IO, WebRTC	Coding battles, peer rooms, recruiter rooms, live communication

5. Modules Included in the Project

Module	What It Does
Authentication Module	Register, login, email verification, OTP resend, forgot/reset password, Google login endpoint, JWT protected user access.
Dashboard Module	Shows total interviews, completed sessions, average content score, confidence score, recent interviews, and AI coach shortcut.
AI Mock Interview Module	Generates interview questions by role/topic/difficulty, speaks questions, listens to answers, stores responses, and moves to next question.

Module	What It Does
Coding Interview Module	LeetCode-style coding workspace with problem statement, code editor, language selection, run/submit buttons, and test cases.
Coding Battle Module	Competitive coding practice where users can enter rooms, solve timed problems, and compare results.
Peer Mock Interview Module	Users discover candidates, send invitations, accept/reject requests, enter peer rooms, and submit feedback.
Recruiter Portal	Recruiters schedule interviews, open live rooms, evaluate candidates, score performance, and update recording status.
Resume Module	Resume upload and parsing for personalization and recruiter/candidate context.
Career Roadmap Module	Generates learning roadmaps based on role and weak topics, tracks day-wise completion and progress.
AI Coach Module	Chat-based mentor for career preparation, technical explanation, answer improvement, and fallback guidance.
Admin Module	Admin overview endpoint for platform-level monitoring and management.

Module Interaction Map



Module interaction map showing how the core platform connects to interview, coding, peer, recruiter, roadmap, and coach modules.

6. Face Recognition and Camera Monitoring

Face recognition is implemented on the client side using the face-api.js library. The hook useFaceAnalysis loads TinyFaceDetector, FaceLandmark68Net, and FaceExpressionNet models from the public face-api model source. These models allow the browser to detect whether a face is present, whether multiple faces are visible, and whether expression data can be observed.

The system starts the camera using browser media permissions. Once video is available, face-api.js repeatedly analyzes frames from the video element. If no face is detected, the UI shows No Face Detected. If more than one face is detected, it shows Multiple Faces Detected. If one face is detected, it shows Face

Detected and marks the camera status as valid.

The proctoring hook also runs a frame loop. It uses detection intervals and grace periods so that the system does not instantly punish small movements. For example, no-face and multiple-face conditions are only treated as violations after a short duration. This prevents false warnings when the user briefly moves or the camera frame changes.

- Library used: face-api.js
- Detector: TinyFaceDetector for lightweight browser-based face detection
- Landmarks: 68-point facial landmarks for face positioning and monitoring
- Expressions: FaceExpressionNet for expression-related signals
- Camera source: browser getUserMedia video stream
- Status outputs: Camera Active, Face Detected, No Face Detected, Multiple Faces Detected, Camera Error
- Interview behavior: interview can pause when the face is not detected for a longer duration

7. Voice Recognition, Indian Accent Voice, and Voice Analytics

Voice recognition is implemented using the browser Web Speech API. The hook useSpeechRecognition uses window.SpeechRecognition or window.webkitSpeechRecognition depending on browser support. The recognition language is configured as en-IN, which is suitable for Indian English interview practice.

The speech recognizer runs in continuous mode with interim results enabled. Interim text is shown while the user is speaking, and final transcript chunks are appended to the full transcript. This allows long answers to be captured more reliably than a single short recognition attempt.

The interviewer voice is produced using SpeechSynthesisUtterance. The system attempts to select an Indian English voice when available, prefers en-IN voices, sets language to en-IN, uses a slightly slower speech rate, and adjusts pitch based on selected interviewer gender. This makes the interviewer sound clearer and more suitable for local users.

Voice analytics are calculated from the final transcript. The utility analyzes word count, speaking duration, words per minute, filler words, pause indicators, grammar-like issues, clarity score, fluency score, and confidence score. These metrics are sent with the interview answer so the backend can combine AI evaluation with voice and facial metrics.

- Speech recognition library/API: Browser Web Speech API
- Speech output API: SpeechSynthesisUtterance
- Preferred accent/language: Indian English using en-IN where browser voices support it
- Recognition mode: continuous=true and interimResults=true
- Metrics calculated: WPM, filler word count, pause count, pause duration, clarity, fluency, confidence
- Common filler examples: um, uh, like, you know, actually, basically
- Interview flow: AI speaks first, microphone starts after speaking ends, user response is auto-submitted after a short pause

8. AI Usage in the Project

AI is used for tasks that benefit from language understanding and generation. The backend connects to Gemini AI services through server-side service files. API keys are not exposed to the frontend. This is important because AI keys must remain private and should only be used by the backend.

The AI service generates role-based questions, evaluates candidate answers, creates career recommendations, supports AI coach chat, and generates learning roadmaps. If the AI service is unavailable or quota is exceeded, fallback behavior provides simpler coaching or default output instead of completely breaking the application.

AI Feature	Purpose	Input Used	Output Produced
Question Generation	Create interview questions	Role, topic, difficulty, previous answers	Next question and difficulty
Answer Evaluation	Score response quality	Question, transcript, voice metrics, face metrics	Content and communication feedback
AI Coach	Mentor-style guidance	User question and session context	Coaching answer or fallback advice
Roadmap Generation	Plan learning path	Role and weak topics	Daily learning roadmap
Career Recommendation	Suggest next career steps	Interview report and performance	Recommended skills and projects

9. APIs Included

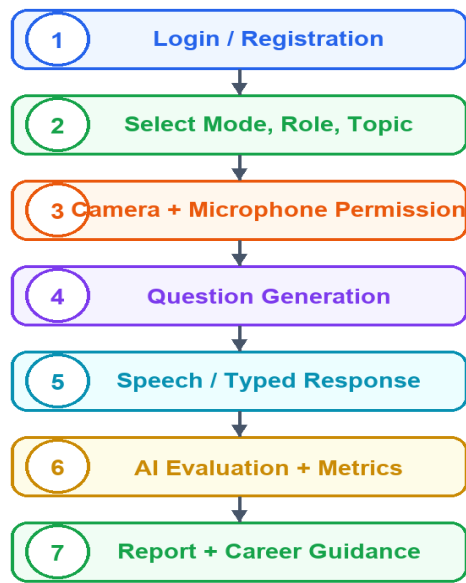
The backend exposes grouped REST APIs. Each group represents one feature area. This makes the application easier to understand and test because authentication, interviews, code execution, roadmaps, peer interviews, recruiter features, resumes, schedules, and admin features are separated.

API Group	Endpoint Examples	Purpose
Auth	/api/auth/register, /login, /google, /verify-otp, /resend-otp, /forgot-password, /reset-password, /me	Account creation, login, verification, password recovery
Interviews	/api/interviews/setup, /:id/answer, /:id/complete, /:id/report, /history/me	AI mock interview setup, response submission, reports
Code	/api/code/run, /api/code/submit	Run and submit coding solutions
Roadmaps	/api/roadmaps/generate, /, /:id, /:id/day/:day/complete	Generate and track learning plans
Coach	/api/coach/sessions, /sessions/:id/message, /sessions/:id/messages	AI coach sessions and messages
Peer	/api/peer/candidates, /invite, /invitations, /room/:roomId, /history	Peer discovery, requests, rooms, feedback
Recruiter	/api/recruiter/schedule, /interviews, /room/:roomId, /score, /recording	Recruiter scheduling and evaluation
Resume	/api/resumes/upload, /my-resumes	Upload and retrieve resumes
Schedule	/api/schedules	Create/list/delete candidate schedules
Admin	/api/admin/overview	Admin overview

10. Database Collections

MongoDB is used because interview data is naturally document-based. A single interview contains setup details, questions, answers, transcripts, scores, face metrics, voice metrics, and final report data. MongoDB and Mongoose allow this nested structure to be stored cleanly.

AI Interview Workflow



Database overview showing important collections and relationships.

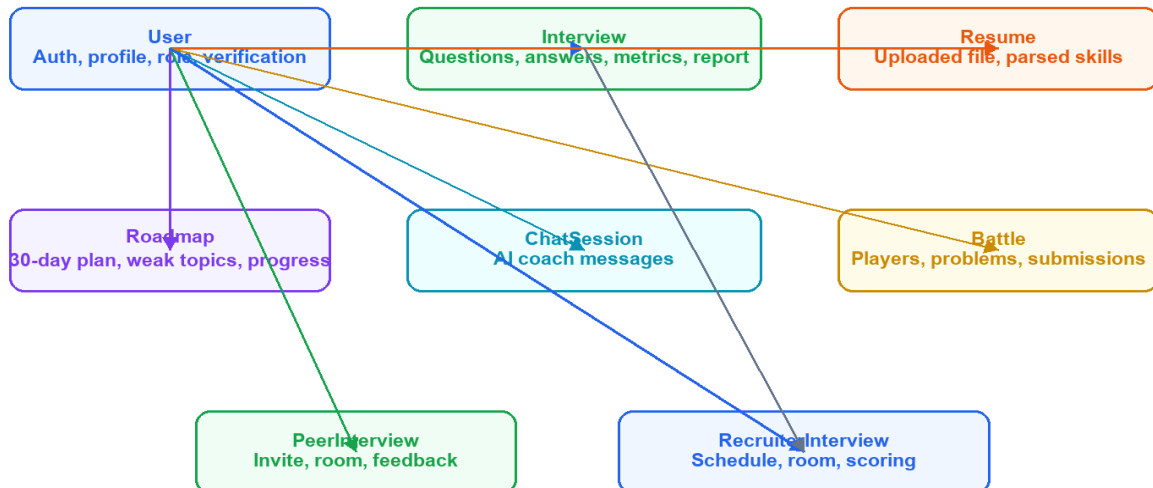
Collection/Model	Stores
User	Name, email, password hash, role, verification state, profile data
Interview	Interview setup, questions, transcripts, AI scores, voice metrics, facial metrics, report
Resume	Uploaded resume metadata and parsed content
Roadmap	Generated learning plan, weak topics, day completion, progress
ChatSession	AI coach conversations and messages
Battle	Battle room, players, coding problem, submissions, result
PeerInvitation	Peer request sender, receiver, status, domain
PeerInterview	Peer room, participants, feedback, history
RecruiterInterview	Recruiter schedule, room ID, candidate, scores, recording status
Schedule	Candidate schedule reminders or interview plans

11. Libraries and Packages Used

Library	Used In	Why It Is Used
react, react-dom	Frontend	Component-based UI development
vite	Frontend build	Fast development and production build
tailwindcss	Frontend styling	Utility-first responsive design
react-router-dom	Frontend routing	Login, dashboard, interview, report, roadmap, peer, recruiter routes
axios	Frontend API calls	HTTP requests to backend APIs
socket.io-client	Frontend realtime	Battle, peer, recruiter room communication
@monaco-editor/react	Coding editor	Professional code editor like VS Code/LeetCode
face-api.js	Face monitoring	Browser face detection, landmarks, expressions
recharts	Reports/dashboard	Charts for performance and analytics
lucide-react	Icons	Consistent UI icons
express	Backend API	REST routes and middleware
mongoose	Backend database	MongoDB schema modeling
jsonwebtoken	Authentication	JWT-based protected routes
bcryptjs	Security	Password hashing
nodemailer	Email	OTP and notification emails
multer	Uploads	Resume upload handling
pdf-parse, mammoth	Resume parsing	Extract text from PDF/DOCX resumes
@google/generative-ai	AI backend	Gemini question generation, evaluation, coach, roadmap
zod	Validation	Request body validation
helmet, cors, morgan	Backend middleware	Security headers, CORS handling, request logging

12. Interview Flow Explanation

Database Schema Overview



AI interview workflow from login to final feedback.

- User logs in using verified credentials.
- User selects interview mode, role, topic, difficulty, and question count.
- System activates camera and microphone permissions.
- AI interviewer speaks the question using speech synthesis.
- After AI speaking ends, microphone listening starts automatically.
- Candidate answers by speech or typing.
- Transcript, voice metrics, and facial metrics are submitted to backend.
- Backend evaluates the answer and generates/stores next question.
- After all questions, the final report is generated and shown to the user.

13. Security and Deployment

The application uses JWT authentication for protected routes and bcryptjs for password hashing. Email verification prevents unverified users from entering the main system. API keys such as Gemini and email credentials are stored in backend environment variables, not in the frontend code.

For deployment, the React frontend is built into static files and served by the Express backend. Render runs the build command, installs dependencies, builds the client, and starts the server. Environment variables such as MONGODB_URI, JWT_SECRET, GEMINI_API_KEY, EMAIL, EMAIL_PASSWORD, CLIENT_URL, and VITE_API_BASE_URL must be configured in Render.

14. Important Results and Observations

- The platform successfully combines multiple interview preparation workflows in one system.
- Face detection and microphone status make the interview experience more realistic.
- Voice analytics provide measurable communication feedback instead of only subjective comments.

- Coding interview and coding battle support technical problem-solving practice.
- Peer and recruiter modules expand the system beyond self-practice.
- Roadmap and AI coach help users continue preparation after the interview report.
- Deployment requires correct build-time frontend variables and backend environment variables.

15. Common Viva / Interview Questions and Answers

Question	Clear Answer
Why did you choose this project?	Because interview preparation is a real student problem. The project also demonstrates full-stack development, AI integration, browser APIs, realtime communication, and deployment.
What is unique in your project?	It combines AI mock interviews, face monitoring, voice analytics, coding interview, coding battle, peer interview, recruiter portal, roadmap, and AI coach in one platform.
How does face recognition work?	The frontend uses face-api.js models to detect faces from the camera stream. It identifies no face, single face, and multiple faces and updates interview/proctoring status.
How does voice recognition work?	The browser Web Speech API listens in en-IN mode, captures interim and final transcripts, and sends the final answer with calculated voice metrics.
What voice metrics are calculated?	Words per minute, filler word count, pause count, estimated pause duration, clarity score, fluency score, and confidence score.
Why is AI used?	AI generates adaptive questions, evaluates answers, supports coaching, creates roadmaps, and produces career recommendations.
How is user data secured?	Passwords are hashed using bcryptjs, sessions use JWT, protected routes require authentication, and sensitive API keys are stored only on the server.
What database is used and why?	MongoDB is used because interview sessions contain nested data such as questions, transcripts, metrics, scores, and reports.
What happens if Gemini quota fails?	The service can return fallback guidance or default content so the application does not completely stop.
How is the project deployed?	The frontend is built with Vite and served from Express. Render hosts the Node server and MongoDB Atlas stores data.

16. Limitations and Future Scope

The project depends on browser support for speech recognition and camera access. Some browsers may not support Web Speech API equally, and AI service quotas can affect generated responses. Free deployment may also introduce cold starts or slower responses.

Future improvements can include multi-provider AI fallback, stronger code execution sandboxing, hidden coding test cases, institution dashboard, recruiter candidate comparison, detailed analytics export, mobile app support, and improved speech recognition accuracy for different Indian accents.

17. Conclusion

The AI-Driven Real-Time Interview Simulation and Recruitment System is a complete project that addresses interview preparation and evaluation through a unified web platform. It includes authentication, AI mock

interviews, speech and face analysis, coding practice, competitive battles, peer interviews, recruiter sessions, resume support, career roadmaps, and AI coaching. The system is practical for students and also demonstrates strong technical coverage for final review.

18. Detailed Module Data Flow

Authentication

User submits registration/login data. Frontend sends request to auth API. Backend validates payload, hashes or compares password, checks verification state, creates JWT, and returns safe user data. MongoDB stores user records and verification state.

AI Interview

User configures interview. Backend creates interview session and gets questions. Frontend displays question, AI avatar speaks, microphone starts, transcript is captured, answer metrics are calculated, backend evaluates and stores response, then next question is selected.

Face Recognition

Camera permission is requested. Video stream is attached to a video element. face-api.js loads TinyFaceDetector, Landmark68, and Expression models. Detection results update UI status and proctoring state.

Voice Processing

Speech recognition captures interim and final chunks. Final transcript is analyzed for WPM, fillers, pauses, clarity, fluency, and confidence. Speech synthesis speaks questions using Indian English preference when available.

Coding Interview

Problem details are shown with constraints and examples. Candidate writes solution in editor. Run checks examples, submit records final attempt. Face monitoring remains active during coding mode.

Peer Interview

User searches candidates, sends invitation, receiver accepts, room opens, users practice together, feedback is saved for history.

Recruiter Interview

Recruiter schedules room, candidate joins link, recruiter views room and submits scores/feedback, result is stored for later review.

Roadmap and Coach

User gives weak topics or asks a question. Backend calls AI services. Roadmaps and coach sessions are stored so the user can continue later.

19. What To Say If Faculty Asks About Face Recognition

The face recognition part is not biometric identity verification. It is interview monitoring. The system checks whether a candidate face is visible, whether multiple faces are detected, and whether camera status is stable. This is useful because in online interviews the candidate should remain visible and focused.

- face-api.js runs in the browser, so raw video does not need to be continuously uploaded to the server.
- TinyFaceDetector is selected because it is lightweight and suitable for real-time browser detection.
- FaceLandmark68Net helps identify facial landmarks and supports position-related checks.
- FaceExpressionNet supports expression-related observations where available.

- Grace periods are used to reduce false warnings when the candidate moves briefly.

20. What To Say If Faculty Asks About Voice Recognition

The voice part has two sides: speech recognition and speech synthesis. Speech recognition converts the candidate spoken answer into text. Speech synthesis allows the AI interviewer to speak questions aloud. The language is configured around Indian English so the experience is more comfortable for local candidates.

- Browser Web Speech API is used for recognition; Chrome/Edge support is recommended.
- `continuous=true` keeps listening for longer answers.
- `interimResults=true` shows live partial text while speaking.
- `en-IN` improves suitability for Indian English.
- `SpeechSynthesisUtterance` is used for AI interviewer voice output.
- Voice metrics convert transcript and duration into communication scores.

21. Complete Technology Justification

Area	Technology	Reason
Frontend	React + Vite	React provides reusable components and Vite provides fast development/build output.
Styling	Tailwind CSS	Fast professional UI design with responsive utility classes.
Backend	Node.js + Express	Lightweight API server for authentication, interviews, AI, code, peer, recruiter, and roadmap routes.
Database	MongoDB + Mongoose	Flexible documents for nested interview responses, metrics, reports, and roadmap progress.
AI	Gemini API	Used for question generation, answer evaluation, coaching, and roadmap generation.
Realtime	Socket.IO + WebRTC	Supports live rooms, battle state, peer interviews, and recruiter sessions.
Security	JWT + bcryptjs	JWT protects routes and bcrypt hashes passwords.
Email	Nodemailer	Sends OTP verification and password reset messages.
Resume	Multer + pdf-parse + mammoth	Handles upload and text extraction from PDF/DOCX resumes.

22. Interview Explanation Script

If asked to explain the project from start to end, use this flow:

- First, the user registers and verifies email through OTP.
- After login, the dashboard shows performance summary and navigation to all modules.
- For AI interview, user selects role/topic/difficulty and starts the live room.
- The camera and microphone are activated; face recognition verifies candidate visibility.

- The AI interviewer speaks questions; speech recognition captures candidate responses.
- Voice analytics and facial metrics are sent with answer text for evaluation.
- The backend stores responses and generates a report with scores and recommendations.
- For coding, the user solves problems in a code editor with test cases.
- For peer and recruiter modules, real-time rooms support collaborative or professional interviews.
- Roadmap and AI coach provide preparation after the interview.

23. Possible Faculty Questions About Deployment

- Why did deployment fail earlier? Vite/Rollup optional dependency and frontend API URL environment variables needed correct setup.
- What is VITE_API_BASE_URL? It is a build-time frontend variable that tells React where the backend API is hosted.
- What is CLIENT_URL? It is used by backend CORS and email URL generation to know the allowed frontend origin.
- Why clear build cache? Vite injects environment variables during build, so old builds may still contain localhost URLs.
- Why serve React through Express? It allows one Render web service to host both frontend and backend together.

24. Final Strengths of the Project

The strongest point of this project is that it is not limited to one feature. It combines real interview simulation, measurable communication analytics, coding practice, peer collaboration, recruiter evaluation, AI coaching, roadmap generation, and deployment readiness. This makes the project useful for students preparing for placements and also technically strong for academic evaluation.